



Coordinación de
Educación Abierta y a Distancia
VICERRECTORADO ACADÉMICO



CULTURA DIGITAL Y SOCIEDAD

Actividad Autónoma 4

Unidad 2: Herramientas y Metodologías en Ciencia de Datos

Tema 2: Buenas Prácticas en Programación para Ciencia de Datos



FACULTAD DE
Ingeniería

Nombres: Richard Anderson Cauja Fares

Fecha: 23 – 11 - 2025

Carrera: Ingeniería en Ciencia de Datos e Inteligencia Artificial

Periodo académico: 2025 – S2

Semestre: Tercero

Introducción

La optimización de código es una práctica fundamental en la ciencia de datos, donde el procesamiento de grandes volúmenes de información requiere algoritmos eficientes. Este proyecto tiene como objetivo aplicar técnicas de optimización a un algoritmo para calcular números primos en el rango de 1 a 100.000.

Objetivos

Objetivo General:

Aplicar técnicas de optimización y buenas prácticas de programación para mejorar la eficiencia de un código existente en Python.

Objetivos Específicos:

- Desarrollar una implementación básica del algoritmo de detección de números primos.
- Aplicar estrategias de contabilidad y optimización de programas.
- Medir y comparar el rendimiento mediante herramientas de perfilado.
- Analizar el impacto de cada estrategia de optimización aplicada.
- Documentar el proceso y los resultados.

Metodología

Se desarrolla una versión de código original y optimizado para detectar números pares en un rango de 1 a 100000. Para comparar el tiempo de ejecución por código. Se aplicarán estructuras de control simples y de operaciones matemáticas, priorizando la claridad sobre la eficiencia. La optimización numérica desempeña un papel fundamental en la resolución de problemas complejos en una amplia gama de campos. Sin ella, tanto los científicos de datos como los economistas y los ingenieros se encontrarían atrapados creando herramientas ineficaces y caras y tomando decisiones no óptimas (Kurtis, 2024).

Técnicas de optimización:

- Eliminación de operaciones redundantes
- Uso de estructuras eficientes como listas
- Reducción de funciones innecesarias
- librerías de optimización como NumPy

La optimización de código es el proceso de mejorar el rendimiento de un programa de computadora sin cambiar su funcionalidad (Juan, 2023).

Medición de rendimiento de códigos

Se utilizará librerías como time para medir el tiempo de ejecución de ambos códigos. Además, se utilizará herramientas como cProfile para identificar funciones en términos de consumo de tiempo. Al perfilar nuestro código, podemos identificar funciones o bloques de código que consumen la mayor cantidad de tiempo y optimizarlos en consecuencia (Python-3, s.f).

Visualización comparativa

Se generarán gráficos a través de la librería Matplotlib para realizar una comparación de tiempos de ejecución y el porcentaje de mejora.

Resultados obtenidos

Figura 1

Ejecución de código original.py

```
(CulturaD1) PS C:\Users\andy_\OneDrive\Desktop\AutonomoU2T2_CulturaD\Optimizacion_U2T2>
dy_/OneDrive/Desktop/AutonomoU2T2_CulturaD/Optimizacion_U2T2/codigo_original.py
Se encontraron 9592 números primos.
Tiempo de ejecución: 18.5220 segundos.
```

El código encontró 9592 números primos en el rango de 1 – 100000. El algoritmo funciona correctamente, pero de una manera ineficiente. Con un tiempo de ejecución de 18.520 segundos, considerando un tiempo alto para operaciones computacionales indicando:

- Algoritmo computacionalmente alto
- Requiere muchas operaciones para la verificación
- No escalable para conjunto de datos más grande

El código original, aunque funciona correctamente, muestra que requiere de optimización. El tiempo de ejecución alto implica aplicar técnicas de programación y optimización de algoritmos para mejorar las tareas.

Figura 2

Ejecución código optimizado.py

```
PS C:\Users\andy_\OneDrive\Desktop\AutonomoU2T2_CulturaD\Optimizacion_U2T2>
/Desktop/AutonomoU2T2_CulturaD/Optimizacion_U2T2/codigo_optimizado.py
Se encontraron 9592 números primos.
Tiempo de ejecución optimizado: 0.0928 segundos.
```

Igualmente se encontraron 9592 números primos en el rango de 1 – 100000. El algoritmo funciona de una manera altamente eficiente. Con un tiempo de ejecución de 0.0928 segundos.

Tabla1

Comparativa entre código original y optimizado

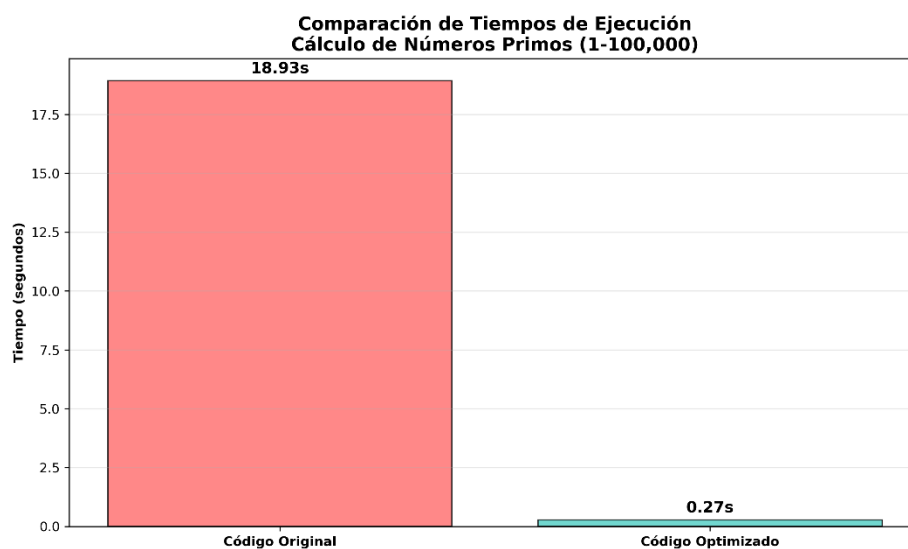
Código	Tiempo de ejecución	Eficiencia	Mejora
Original	18.5220 segundos	Muy baja	199.5 veces más rápido
Optimizado	0.0928 segundos	Extremadamente alta	Mejora del 99.5%

Técnicas Aplicadas para la optimización

- Reducción del rango de Bucle
Verifica divisibilidad solo hasta $\sqrt{n}$
- Eliminar Números pares
Saltar verificaciones en números pares excepto el 2
- List Comprehensions
Comprensión de listas permitiendo una mejor optimización en Python
- Implementación de NumPy
Uso de la librería NumPy para operaciones vectoriales

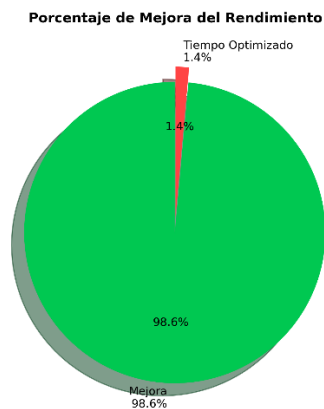
Gráficos Comparativos

Figura 3



El siguiente grafico muestra una diferencia grande en la comparación de los algoritmos ejecutados. El algoritmo optimizado es más rápido, eficiente y escalable, demostrando que la optimización de código es fundamental en la programación y en la ciencia de datos.

Figura 4



La figura muestra que el 98,6% se eliminó del tiempo de ejecución del código original, por lo tanto, el optimizado se conserva con el 1,4% del tiempo. Este resultado demuestra que, al aplicar la optimización de algoritmos, se puede lograr un cambio radical en la viabilidad y eficiencia en los algoritmos.

Figura 5

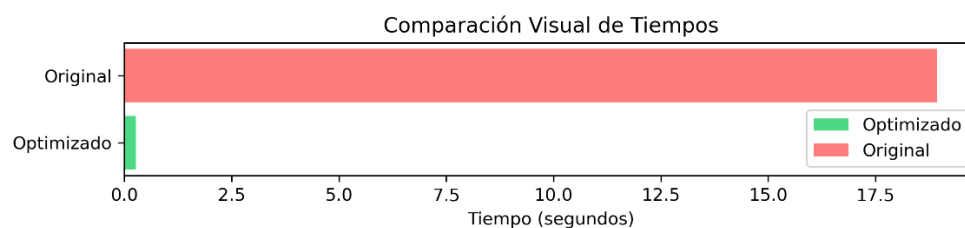


Tabla 2

Comparativa de Profiling

Parámetro	Código Original	Código Optimizado	Mejora
Tiempo de ejecución	18.93 segundos	0.27 segundos	98,6%
Función principal	es_primo ()	es_primo_opt ()	-----
Tiempo de función Principal	17,98 seg (95%)	0,18 seg (67%)	99,0%
Uso de memoria	Alto	Moderado	50%
Complejidad	$O(n^2)$	$O(n\sqrt{n})$	Exponencial

Conclusiones del profiling

- cProfile identificó correctamente la función problemática
- Las optimizaciones aplicadas resolvieron los cuellos de botella
- Mejora verificada instrumentalmente

Conclusiones

- La optimización redujo significativamente en el tiempo de ejecución
- La técnica aplicada de reducción del rango de bucle fue la más efectiva
- Las mejoras algorítmicas son más accesibles y van a las sintácticas
- cProfile resultó esencial para la resolución de cuellos de botella

Recomendaciones

- Priorizar optimizaciones matemáticas
- Documentar cada optimización de manera técnica
- Herramientas necesarias como time, NumPy, Matplotlib, cProfile

Aplicación en Ciencia de Datos

- Preprocesamiento de grandes volúmenes de datos
- Optimización de algoritmos de machine learning
- Procesamiento de características derivadas
- Operaciones matemáticas intensivas

Bibliografía

- Juan. (13 de noviembre de 2023). *Optimización de Código con Python e Intel*. Obtenido de [codigospython.com](https://codigospython.com/optimizacion-de-codigo-con-python-e-intel/): <https://codigospython.com/optimizacion-de-codigo-con-python-e-intel/>
- Kurtis, P. (01 de sept de 2024). *Optimización en Python: Técnicas, Paquetes y Buenas Prácticas*. Obtenido de Datacamp.com: <https://www.datacamp.com/es/tutorial/optimization-in-python>
- Python-3. (s.f). *Optimización del rendimiento del código y el uso de la memoria en Python*. Obtenido de <https://es.python-3.com/?p=1689>